Type-checking in LAMBDA

How the implementation represents and checks types

CS 413

October 20, 2026

What this implementation checks

- Lambda and recursive-function parameters carry type annotations.
- A let-bound name gets its type from its initializer; its annotation is optional.
- The parser stores annotations and let initializers in the expression tree.
- A recursive checker walks the tree and checks each operation.
- A small unifier connects unknown element types for empty lists and options.

The central question

Given the expression tree and the types of names in scope, can every part of this expression be used consistently?

Types are written at binders

```
let
  inc =
    lam (n : int) => n + 1
  answer = inc(41)
in
  answer
end
```

The lambda annotation gives the parameter type. Each let-bound name gets the type of its initializer, so a let annotation is optional.

Names are introduced explicitly; lambda and recursive-function parameters still carry annotations.

The type vocabulary

$$T ::= \text{int} \mid \text{bool} \mid T \to T \mid (T_1, \ldots, T_n)$$
$$\mid \text{List}[T] \mid \text{Optn}[T]$$

- Function: `int -> bool`
- Tuple: `(int, bool)`
- Empty tuple: `unit`
- List and option: `List[int]`, `Optn[bool]`

Lists and options are homogeneous: every element or wrapped value has the same type.

Types have matching data structures

The checker represents types as Python objects:

Type object	Meaning
`TInt()`	integer
`TBool()`	boolean
`TFun(argument, result)`	function type
`TTuple(items)`	tuple with component types
`TList(element)`	homogeneous list
`TOption(element)`	option value
`TMeta(id)`	temporary unknown type

For example, `int -> List[bool]` is `TFun(TInt(), TList(TBool()))`.

The AST keeps each annotation

The parser turns source into dataclass nodes. The checker reads their fields.

Lambda

`DOElam(arg1, arg2, ann)`

- `arg1`: parameter name
- `arg2`: body
- `ann`: parameter type

Recursive function

`DOEfix(arg1, arg2, arg3, ann, parameter_type)`

- function name and parameter name
- body
- declared function and parameter types

Let declarations store their initializer and an optional declared type.

Parsing annotations

The parser handles types with a small recursive-descent routine:

1. Read a type atom: `int`, `bool`, a tuple, `List[T]`, or `Optn[T]`.
2. If an arrow follows, parse the result type recursively.
3. Build a `TFun` node. Recursion makes arrows associate right.

Right-associative arrows

`int -> bool -> int` means `int -> (bool -> int)`.

Malformed syntax is rejected before type-checking begins. Let declarations may omit their annotation; lambda and recursive-function parameters still require theirs.

The environment is a type lookup table

$$\Gamma = \{\texttt{inc} : \text{int} \to \text{int}, \quad \texttt{xs} : \text{List[int]}\}$$

- Checking a variable looks up its name in Γ.
- Checking a lambda adds its parameter and annotation for the body.
- Checking a let initializer uses the current environment.
- After the initializer checks, its type (or the declared type, if present) enters the environment for later declarations.

Sequential scope

In `let x = 4 y = x + 1 in y end`, the first `x` is available in the initializer for `y`; its type is inferred as `int`.

The checker follows the expression tree

The nested `check(expr, env)` function handles one AST form at a time.

Examples

- integer literal returns `TInt`
- variable returns its environment type
- addition checks two integers
- comparison checks two integers and returns boolean

Conditional

$$\frac{c : \text{bool} \quad a : T \quad b : T}{\text{if } c \text{ then } a \text{ else } b : T}$$

The implementation checks the condition, checks both branches, then unifies their types.

Function application creates a type constraint

For an application `f(arg)`, the checker:

1. checks the type of `f`
2. checks the type of `arg`
3. requires `f` to have type `argumentType -> resultType`

A valid call

If `inc : int -> int`, then `inc(41) : int`.

A rejected call

`inc(true)` creates the constraint `int = bool`, which cannot be solved.

Unification fills in unknown element types

`list_nil()` has no element to inspect, so the checker gives it a fresh type variable, say α.

$$\texttt{list_nil()} : \text{List}[\alpha]$$

In `list_cons(4, list_nil())`, the integer head constrains α to `int`. The result is `List[int]`.

- `TMeta(id)` represents a fresh unknown.
- Unification records substitutions from unknowns to known types.
- An occurs check prevents circular types such as a type being a list of itself.

The same mechanism handles `optn_nil()` and the result of `list_head`.

Why unification needs an occurs-check

When solving a constraint $\alpha = T$, it is not enough to ask whether T is a type. We must also make sure T does not contain α.

A circular constraint

Suppose application constraints lead to $\alpha = \alpha \to \text{int}$. Substituting this back into itself gives

$$\alpha = (\alpha \to \text{int}) \to \text{int} = (((\alpha \to \text{int}) \to \text{int}) \to \text{int}) = \cdots$$

No finite type satisfies this equation.

The occurs-check rejects a binding if the unknown already appears inside the type assigned to it.

How the occurs-check works

To bind metavariable α to type T:

1. First apply current substitutions to T.
2. Recursively search T's structure for α.
3. If found, report a type error; otherwise record $\alpha \mapsto T$.

```
def bind(meta, typ):
    typ = apply_substitution(typ)
    if typ == meta:
        return
    if occurs(meta, typ):
        raise TypeCheckError("infinite type")
    substitution[meta] = typ
```

For a function type, `occurs` searches both argument and result types; for lists, options, and tuples, it searches their component types.

Checking recursion

```
fact : int -> int =
  fix fact(n: int): int =>
    if n <= 1 then 1 else n * fact(n - 1)
```

The checker verifies three things:

1. The recursive name `fact` has the declared type `int -> int` inside its body.
2. The parameter `n` has type `int`.
3. The body has the declared result type `int`.

A recursive call is checked like any other function application.

A worked example from the test programs

```
apply_twice : (int -> int, int) -> int =
  lam (p : (int -> int, int)) => p.0(p.0(p.1))
```

The environment inside the body contains:

$$\texttt{p} : (\text{int} \to \text{int}, \text{int})$$

So projection `p.0` has type `int -> int` and `p.1` has type `int`. Applying the function twice still returns `int`, which matches the annotation.

Type errors point to the violated constraint

The examples in `LAMBDA/mytest/type_error_*.lam` isolate common failures:

- `is_small(true)`: expected integer argument, got boolean.
- `if 1 then ...`: condition must be boolean.
- `if true then 10 else false`: branch types disagree.
- `list_cons(true, list_cons(1, ...))`: list elements disagree.
- Recursive body returns boolean where its annotation promises integer.

The exception is `TypeCheckError`, a subtype of Python's `TypeError`.

Using the checker

```
from pathlib import Path
from LAMBDA.lambda2 import d0cls_parse, d0cls_typecheck
from LAMBDA.lambda2 import d0exp_parse_and_typecheck

decls = d0cls_parse(Path('LAMBDA/mytest/examples.lam').read_text())
d0cls_typecheck(decls)

expr, typ = d0exp_parse_and_typecheck('1 + 2')
```

For a complete program, call the checker before the evaluator. The evaluator retains dynamic checks, but does not invoke the static checker automatically.

Scope and limits

- The checker is monomorphic: each let-bound name has one type.
- Let-bound names take the initializer's type; lambda and recursive-function parameters remain annotated.
- Empty list and option types are solved from their uses.
- `list_head`, `optn_get`, and division can still fail at runtime on empty values or a zero divisor.
- Type-checking does not prove termination or program intent.

Takeaway

The implementation turns type rules into a recursive AST walk, with unification where an element type is initially unknown.

Let types come from their initializers

Current behavior

Let declarations may omit a type annotation. The checker uses the initializer's type for the new name. If a let annotation is supplied, the initializer must match it. Lambda parameters and recursive-function parameters remain annotated.

The existing metavariables infer selected unknowns, such as the element type of an empty list or option; let-bound names still receive the resolved type of their initializer.

Possible extension: infer parameter types

The current parser requires annotations on lambda and recursive-function parameters. To infer those types, the parser could record omitted annotations as absent and the checker could supply fresh unknown types.

Binder	Unknown introduced	Use
`lam (x) => e`	$x : \alpha$	infer function type $\alpha \to \tau$
`fix f(x) => e`	$x : \alpha,\ f : \alpha \to \beta$	check body against β

The AST can keep the same shape with optional parameter annotation fields. A missing annotation means "create an unknown," not "skip checking."

Constraints arise from uses

For each expression, return a type that may contain unknowns; operations add equalities that the solver must satisfy.

```
lam (x) => x + 1
```

Initially $x : \alpha$. Addition requires $\alpha = \text{int}$, so the result is int and the whole expression has type $\text{int} \to \text{int}$.

```
lam (f) => lam (x) => f(x)
```

Application requires $f : \alpha \to \beta$ and $x : \alpha$. The inferred type is $(\alpha \to \beta) \to \alpha \to \beta$.

An unused parameter stays unknown: `lam (x) => 0` has type $\alpha \to \text{int}$ for any α.

One solver, two phases

1. **Generate constraints:** walk the AST, create fresh metavariables, and unify types required by each construct.
2. **Solve and report:** apply substitutions to the expression's type; report a mismatch where unification fails.

For application $e_1(e_2)$, infer $e_1 : \tau_1$ and $e_2 : \tau_2$, choose fresh β, then unify τ_1 with $\tau_2 \to \beta$. The result type is β.

Implementation detail

Metavariables must be read through the current substitution before they are compared or printed. The occurs check still prevents infinite types.

Let and recursion need special care

Monomorphic let

```
let id = lam (x) => x
in (id(1), id(true)) end
```

With one shared type for `id`, the calls force its argument type to be both `int` and `bool`; this is rejected.

Recursive binding

- Give `f` a provisional type $\alpha \to \beta$ before checking its body.
- Check recursive calls using that same provisional type.
- Unify the body result with β; check any declared annotation too.

Polymorphic let requires an extra generalization step: quantify unknowns not fixed by the surrounding environment, then instantiate them freshly at each use. That is a further extension beyond monomorphic inference.

What is a REST API?

An API is a contract that lets one program request services from another. A REST-style web API exposes resources at URLs and uses familiar HTTP operations to work with them.

HTTP method	Typical meaning
`GET`	retrieve a resource
`POST`	submit data or ask the server to perform an operation
`PUT/PATCH`	replace or update a resource
`DELETE`	remove a resource

The URL identifies what the request is about; the method says what action is intended. Requests and responses commonly carry JSON data.

Resources and their representations

A **resource** is the thing an API lets a client refer to. A **representation** is data that describes the resource or carries its state in a particular format.

Resource: a LAMBDA program

```
{"source": "1 + 2"}
```

This JSON is a representation that carries the program text to the service.

Compile result:

```
{"ok": true,
 "type": "int"}
```

This JSON represents the result of checking that program.

The representation is not the resource itself; it is a transferable description of its state or result.

REST: requests are self-contained

Each request carries the information the server needs to handle it, such as the URL, method, headers, and body. The server does not rely on hidden conversation state from an earlier request.

- The client and server communicate through a uniform HTTP interface.
- Responses include a status code and may include a JSON body.
- A client can be a browser, command-line tool, or another service.

One request, one answer

Send source code in a request body; receive a result and diagnostics in the response body.

REST is an architectural style, not a specific programming language, framework, or wire format.

Map the compile service to HTTP

The compiler service has one main job: check a submitted LAMBDA program.

Part	Example	Role
Method	`POST`	submit source for checking
Path	`/api/compile`	name the compile operation
JSON body	`"source": "1 + 2"`	carry the program text
JSON response	`"ok": true, "type": "int"`	return result

The client sends a request; the service parses and type-checks the source, then returns a response. No client session is needed for this basic operation.

From compiler library to web service

The parser and checker can sit behind a small HTTP API:

$$\text{Browser or client} \longrightarrow \text{HTTP service} \longrightarrow \text{LAMBDA front end}$$

- Client submits LAMBDA source as text.
- Service parses and type-checks it, then returns structured JSON.
- Keep the compiler functions independent of HTTP; the route is a thin adapter.

First endpoint

`POST /api/compile` accepts one program and reports whether it is well-typed, along with its type or diagnostics.

Define a request and response

Request: `POST /api/compile`

```
Content-Type: application/json

{"source": "1 + 2"}
```

Successful response: `200`

```
{
  "ok": true,
  "type": "int"
}
```

For invalid programs, return `ok: false` and a diagnostic. Use 400 for malformed JSON or missing fields; reserve 500 for unexpected server failures. Keep response fields stable so clients can handle errors reliably.

The route is a narrow adapter

```
def compile_route(request):
    source = require_string(request.json, "source")
    try:
        decls = d0cls_parse(source)
        typ = d0cls_typecheck(decls)
        return {"ok": True, "type": format_type(typ)}, 200
    except (ParseError, TypeCheckError) as err:
        return {"ok": False, "error": str(err)}, 200
```

The exact parser return shape and checker API determine the glue code. Validate input size, avoid exposing stack traces, and do not evaluate submitted programs unless execution is an explicit feature with its own limits.